

FaizOS: a teaching system that rewrites itself from one learner's answers

Everything except the curriculum: how lessons are made, checked and delivered, where every rule lives and why, the loops that turn Faiz's answers and complaints into a better method, and the machinery (hooks, scripts, agents, an MCP server and a database) that keeps it honest.

1 rulebook 39 rules he stated (C1 to C39) 48 measured findings 34 taught words
5 hooks 5 scripts 2 agents 39 MCP tools 20 DB tables 18 commands
52 self-checks

[Principles](#) [Map](#) [Layers](#) [Lesson lifecycle](#) [Learning loops](#) [The style](#) [Questions](#) [The build](#)
[Enforcement](#) [Machinery](#) [Constraints](#)

01 Design principles

Six ideas that every other part of the system serves. Each one exists because an earlier version broke without it.

One source of instructions

All teaching rules live in one file, the faiz-teach skill. Memory notes, the insights table, commands and docs point at it and hold no rules. Seven conflicting sources caused the "inconsistent method" complaint of 14 Sep.

Evidence and instructions never mix

`learning-evidence.md` holds only what happened (counts, quotes, the ledger). The skill holds only what to do. Every rule cites the evidence ID or date behind it.

His words win

A rule he states changes the rulebook in the same turn, replacing the old line in place. Research loses to his record whenever they disagree.

Nothing unchecked reaches him

Every teaching part is written to a script, passes a checker, and is sent word for word. A Stop hook blocks any teaching message that is not in a validated script.

Read and judge, never write

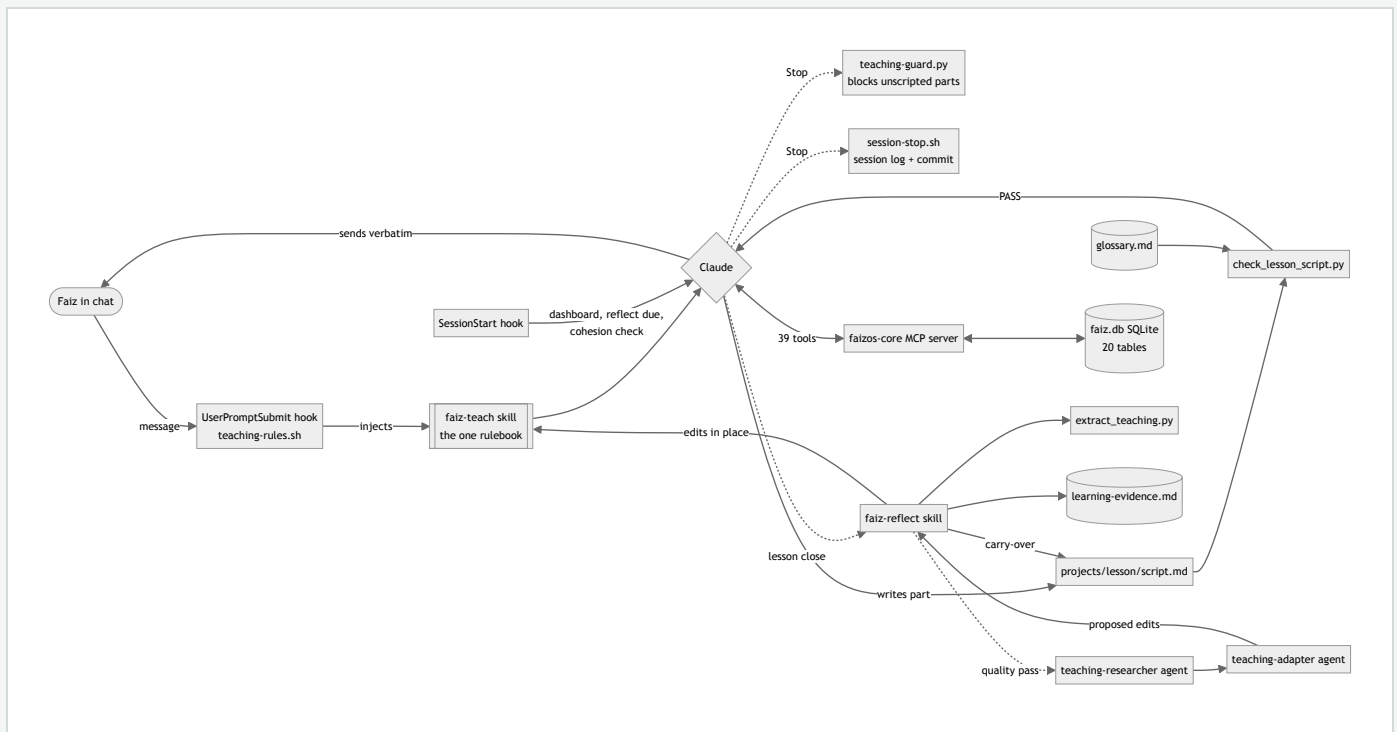
He reads code, predicts what it does, finds planted faults and makes engineering decisions. Claude writes and runs the code he chooses (C30, 12 Sep).

Measured, not claimed

Progress bars count database rows. Builds report real output. Estimates are labelled as estimates and corrected in the open when wrong.

02 System map

What talks to what. Solid arrows run every time; dotted arrows run at lesson close or on request.



03 The five layers

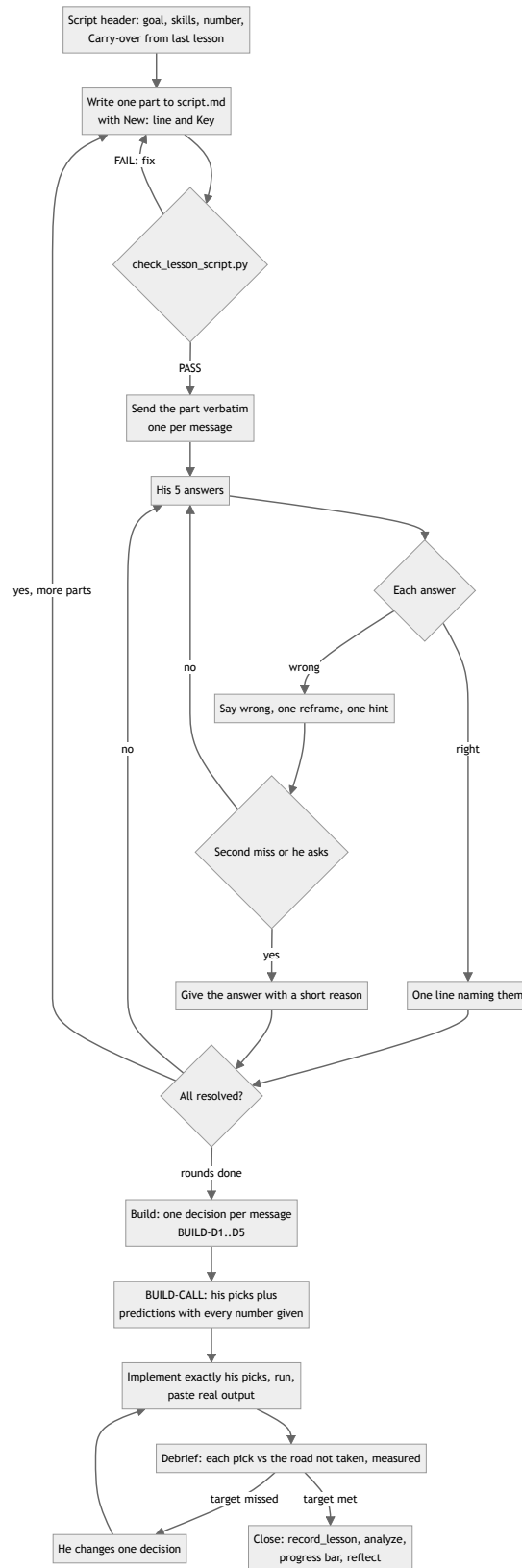
Each layer has one job and one home, so no two places can disagree.

LAYER	HOME	HOLDS	CHANGED BY
Instruction	.claude/skills/faiz-teach/SKILL.md	Who he is, lesson shape, part template, build format, question bank, how answers are marked, words, delivery, how the file itself changes. 183 lines.	His stated rules (same turn); patterns in 2+ parts; the adaptor's evidence-backed edits

LAYER	HOME	HOLDS	CHANGED BY
Evidence	docs/learning-evidence.md	Tables A (formats measured), B (message features that predict understanding), C (his 39 rules, with status), D (what recovers him when stuck), E (recurring weak spots), the Session ledger, and the taught list.	The reflect step only
Enforcement	scripts/, hooks/, docs/glossary.md	The part checker, the Stop guard, the jargon gate, the system cohesion check.	Kept in step with the rulebook's marker lines
State	faizos-core/faiz.db, notebook/, memory/	Skills and mastery, lessons, errors, builds, experiments, streak; the session log; a one-line pointer in memory.	MCP tools (additive only); session hooks
Improvement	.claude/skills/faiz-reflect, .claude/agents/, docs/research/	The reflect procedure, the two agents, the research digest and adaptation.	Lesson close, his corrections, the quality pass

04 A lesson, end to end

Two rounds of parts, one decision build, then close. Each step is gated by the one before it.



05 The learning loops

Six loops at six speeds. The fast ones fix a single answer; the slow ones change the method itself. Each ends by writing somewhere the next loop reads.

1 • The answer loop EVERY ANSWER • SECONDS

his answer → mark against the Key → wrong: reframe + one hint →
stuck ladder → resolved

The stuck ladder is ordered by what has worked for him: first point at his own earlier answer (10 of 12, D1), then one new everyday picture (8 of 10) or a two-option question (5 of 7). Never re-explain in longer prose (0 of 5, D6). After a second failed hint, or when he asks, give the answer with a reason. A given answer returns once, reworded, in a later part.

2 • The correction loop EVERY COMPLAINT • SAME TURN

his words → new C row, quoted → rulebook line replaced in place →
checker updated if mechanical → commit

Examples: "explain the jargon you throw" became the glossary gate (C33); "decisions cannot be this obvious" became the trade-off check (C35); "Idw predict, just build" (C37) was superseded the same day by C39 ("I don't mind predicting if I'm given all the data"). He never has to say a thing twice.

3 • The reflect loop EVERY LESSON CLOSE

extract Q&A from transcript → score first-try by type → ledger row + tables →
rule change if 2+ parts show it → Carry-over into next script →
mark reflected

`extract_teaching.py` pairs each question message with his reply and remembers where it stopped (`reflect-state.json`). SessionStart warns "UNREFLECTED TEACHING" if a session taught without reflecting. The next lesson's script is refused by the checker until it carries a `Carry-over from L<n>:` line, so lessons cannot forget what the last one learned.

4 · The quality pass

EVERY 3RD LESSON · OR ON REQUEST

teaching-researcher → graded digest, 29 sources → teaching-adapter →
"replace this line / with this" → apply what his evidence supports

The researcher reads cognitive-science research (Chi, Sweller, Mayer, Roediger, Bjork, Kapur, Schwartz, Renkl) and grades each claim strong, moderate or weak. The adapter checks it against his own numbers, keeps every rule he stated, and proposes edits plus a before-and-after rewrite of a real part. The researcher re-runs when its digest is over 60 days old.

5 · The mastery loop

EVERY LESSON CLOSE

first-try accuracy per skill → faizos_analyze → mastery in DB →
progress bar → teach_next gap

Mistakes are classified into an error taxonomy (errors table) that the drill command targets. The progress bar counts rows, never estimates: lessons done, production skills touched, capstone rungs backed by a real metric.

6 · The session loop

EVERY SESSION

SessionStart: dashboard + reflect status + cohesion check → work →
Stop: session summary from DB facts, commit

The session log is written deterministically, with no model involved, so it cannot be skipped or embellished.

06 The style: how a part is built

Every teaching part, including parts about real lesson files, has this anatomy. Parts built this way scored 24 of 30; parts that skipped it scored 3 of 11 on the same day (A12, A13).

The problem.

4 to 6 sentences: who uses this and why, how it works today, what goes wrong, the mechanism in physical terms, what it costs. Every rate also as counts ("92% (55 of 60)"). Banned: phrases that hide the mechanism ("happen to", "somehow").

The fix:

One sentence, plain words.

Tiny example

8 lines or fewer, generic names, only known pieces plus the one new thing.

How it works	Only for a formula or a measure: each piece on its own line, worked on one real case, then what it means for the business. Raises the size limit to 3,200 characters.
Picture:	One everyday picture that runs on the same mechanism, plus "Where it breaks".
Paths	Every path as a bullet: if this, these lines run; if that, this is skipped.
Worked chain	Before any "which way does it bend" question: wrong input → number moves → decision bends → cost, on another case. Faded once he gets one right cold.
Your turn.	Five questions (see next section).
### Key	Never sent. Each answer tagged with its question type, plus "Relies on:" listing every rule the questions use, each already taught.

Words

Plain vocabulary from his bootcamp: sticker (variable), machine (function), slot (parameter), kind, piling up. Real Python names are kept. Any word on the watchlist must be taught already or explained in the same sentence, or the checker fails the part.

One new thing per part

Count every new keyword, machine and domain word. More than one: split the part. A new measure gets one counted case from the part's own numbers.

No repetition

Never re-teach what he got right. "ik this" means move on now. No recaps of the plan, his progress or these rules. Full detail on new ideas, one line on known ones, nothing on mastered ones: the research explains why (expertise reversal).

Delivery

One part per message, then wait. Real lesson code is pasted in chat, at most 8 lines with notes; never "open the file" or "scroll to line 76" (he rejected a 300-line file unopened).

07

Questions and marking

Five per part: at most one warm-up, at least three from the bank (each needs two or more steps of reasoning), exactly one "Someone broke it". He scores about 88% on plain arithmetic, so it trains nothing new.

TAG	SHAPE	TRAINS
[warmup]	One compute from the part's numbers.	Entry point only

TAG	SHAPE	TRAINS
[chain]	X is off this way. Which way does the number move, which way does the decision bend, what reaches the customer?	E12, direction (0 of 3 in L6)
[prove]	The system reports N. Does that show A, or only the weaker B? What one measurement would show A?	E12
[flip]	At what value of the input does the decision change?	Thresholds, break-even
[pair]	Two setups differ in one thing. Which can you trust, and what number decides it?	Contrasting cases (strongest single method)
[gap]	Which saves more? Give before, after and the gap for each.	E11, one-sided savings
[which-way]	Runs without error. Too high or too low, and who notices first?	E1, quietly wrong
[wrong-step]	A colleague reasoned 1) 2) 3). Which step is wrong?	Misconceptions
[must-be-true]	For the losing option to be right, what would have to be true?	Assumptions
[broke]	A planted change. Crash (it stops), quietly wrong (runs, wrong result), or fine (runs, right result)? Label only.	E1, reading code for faults

Never

- An answer already printed in the part (read-back check)
- A why-question before the mechanism is shown (0 of 4 cold)
- A table to simulate broken code (0 of 4)
- Any hint about an answer's shape
- Write-it-yourself or blank functions (0 of 3)

Marking

- Right: one line naming them
- Wrong: say so, one reframe, one hint, never partial credit plus numbers to plug in
- "I don't understand": stop, rebuild the part with fewer new things
- Next part only when every question is resolved

08 The decision build

Every lesson ends in something real that he builds through decisions, not code (C30). Claude implements exactly his picks, even the ones the Key disagrees with, and the debrief measures

the road not taken.

A decision message

One per message, 3 to 5 per build, 2,000 characters.
The situation with every given stated and its source (model run, his labels, or "assumed:"); unknowns named as unknown. Each option carries:

- What happens
- Effect on the target
- Cost
- Rules out

Ends with **Your pick**.

What makes it hard (C35, C39)

No option rules out nothing; each wins somewhere the others lose; at least two of these are live:

- a given shown as a range
- another team's constraint
- a bad-month effect
- shared blind spots
- hard to undo
- depends on an earlier pick
- measure first

The Key names what the winner gives up and the one given that would flip the pick.

Prediction and run

BUILD-CALL lists his picks and asks for predictions with every needed number printed, at least one combining two picks. The model is written and run before Decision 1 so the givens across decisions cannot clash (they did once, in L6).

Debrief

One line per decision: his pick, what it measurably did, what the other option would have done (run when under 5 minutes, else estimated and labelled).
Target missed: he changes one decision and it runs again.

09 Enforcement: what is checked by machine

A rule that only lives in prose drifts. These run before a part can be sent, when a message ends, and at every session start.

CHECK	WHERE	RULE IT ENFORCES
Template markers present; problem has 4+ sentences; 2+ path bullets; 4 to 6 questions; exactly one Someone broke it with labels defined	check_lesson_script.py	Part template (A12, C34)

CHECK	WHERE	RULE IT ENFORCES
Size: 2,600 characters, 3,200 with How it works, 2,000 for build messages	check_lesson_script.py	B8, C38
Every watchlist word taught or explained in its sentence	check_lesson_script.py + glossary.md	C33
No answer that only reads back a number already in the part	check_lesson_script.py	C36
Answers tagged; at most 1 warm-up, 3+ bank questions, 1 broke	check_lesson_script.py	C38
A direction question needs a Worked chain; a formula needs How it works; vague phrases rejected	check_lesson_script.py	E12, C38
Decision: all option fields twice, no "rules out nothing", an unknown or assumed figure, Key names what it gives up and an "If"	check_lesson_script.py	C31, C35, C39
No "open the file" or "scroll to"; code blocks 10 lines or fewer; New: line lists one thing	check_lesson_script.py	B7, B1, B5

CHECK	WHERE	RULE IT ENFORCES
A script from L5 on needs Carry-over from the last lesson	check_lesson_script.py	Reflect loop
A message asking questions must match a validated script part word for word	teaching-guard.py (Stop)	Delivery (C17)
No stale teaching method in any command, hook, skill, server file or memory note; hooks wired; every script passes	check_teaching_system.py (52 checks)	One source of instructions

10 Machinery

Hooks

- `teaching-rules.sh` (every prompt): injects the rulebook itself, so there is no second copy
- `teaching-guard.py` (Stop): blocks unscripted teaching; fails open, never blocks twice
- `session-stop.sh` (Stop): writes the session summary from DB facts and commits it
- `session-start.sh` : dashboard prompt, reflect status, silent cohesion check
- `pre-tool-guard.sh` (Write/Edit): the older write-from-empty guard, now off on read-and-judge tracks

Scripts

- `check_lesson_script.py` : the part and decision checker; reads its markers from the rulebook
- `extract_teaching.py` : pairs questions with replies; `--mark` records what was reflected
- `reflect_status.py` : counts unreflected exchanges for SessionStart
- `check_teaching_system.py` : the cohesion check
- `install-crons.sh` : venture and frontier fetches, installed only with his go

Agents

- **teaching-researcher**: external research, graded, cited; never edits the rulebook
- **teaching-adapter**: fits the research to his evidence, proposes line-for-line edits, rewrites a real part

Both run on Opus; the reflect skill calls them (step 4b).

MCP server: faizos-core (39 tools)

lesson lesson_start, record_lesson, lesson_progress, analyze, record_insight, curriculum, guidance, mode, track_status

build start_build, spec_build, hint, unlock_build, review_code, record_review, ship, log_experiment, complete_rebuild, rebuilds_due

memory state, progress, list_skills, save_revision, notes, review_queue, reveal_contrast, error_report, cost_drill, config

outward radar_save, radar_list, oss, frontier_ingest, venture_ingest, venture_pending, venture_classify_save, venture_score, venture_activate, venture_review

Database: faiz.db (SQLite, gitignored)

skills · mastery (on skills) · lessons · insights · revisions · reviews · missions · journey_log · meta (streak) · builds · errors · code_reviews · experiments · systems · tracks · frontier · ventures · evidence · radar · cost_drills · oss_targets

Schema changes only through numbered, additive migrations, rehearsed on a verified backup first.

Commands (18)

/faiz · /faiz-learn · /faiz-build · /faiz-spec · /faiz-hint · /faiz-unlock · /faiz-review · /faiz-run · /faiz-analyze · /faiz-errors · /faiz-drill · /faiz-notes · /faiz-cost · /faiz-ship · /faiz-oss · /faiz-radar · /faiz-frontier · /faiz-venture

Each command points at the rulebook for teaching; none carries its own rules.

11 Hard constraints

Never broken, whatever the request

- Never delete, drop, rename or truncate a table, column or row; never modify existing rows in skills, mastery, missions, streak, lessons, insights or revisions. All schema changes are additive.
- The repo is public: `faiz.db` stays gitignored; venture evidence stays local.
- Tests make no network calls. TypeScript strict, no `any` . `npm test` and the smoke test pass before a commit.
- Crontab changes and anything needing his accounts (hosting, GitHub Actions, API keys) happen only with his explicit go, done by him.
- Writing style: plain words, no em dashes, lead with the answer.

- Teaching rules are stored only in the rulebook: never in memory, the insights table or commands.

Sources: `.claude/skills/faiz-teach`, `.claude/skills/faiz-reflect`, `docs/learning-evidence.md`, `docs/research/`, `scripts/`, `hooks/`, `faizos-core/`. Regenerate this page after any quality pass.